# INFORME DE LABORATORIO

| INFORMACIÓN BÁSICA | | | | | |
|---|---|---|---|---|---|
| **ASIGNATURA:** | *Computación gráfica, visión computacional y multimedia* | | | | |
| **TÍTULO DE LA PRÁCTICA:** | *Transformaciones geométricas* | | | | |
| **NÚMERO DE PRÁCTICA:** | *02* | **AÑO LECTIVO:** | *2026-A* | **NRO. SEMESTRE:** | *VII* |
| **FECHA DE PRESENTACIÓN** | *29/04/2026* | **HORA DE PRESENTACIÓN** | *12:20pm* | | |
| **INTEGRANTE (s):**<br>Perez Huamani Jeremy Joshua | | | | **NOTA:** | |
| **DOCENTE(s):**<br>Prof. René Nieto | | | | | |

## SOLUCIÓN Y RESULTADOS

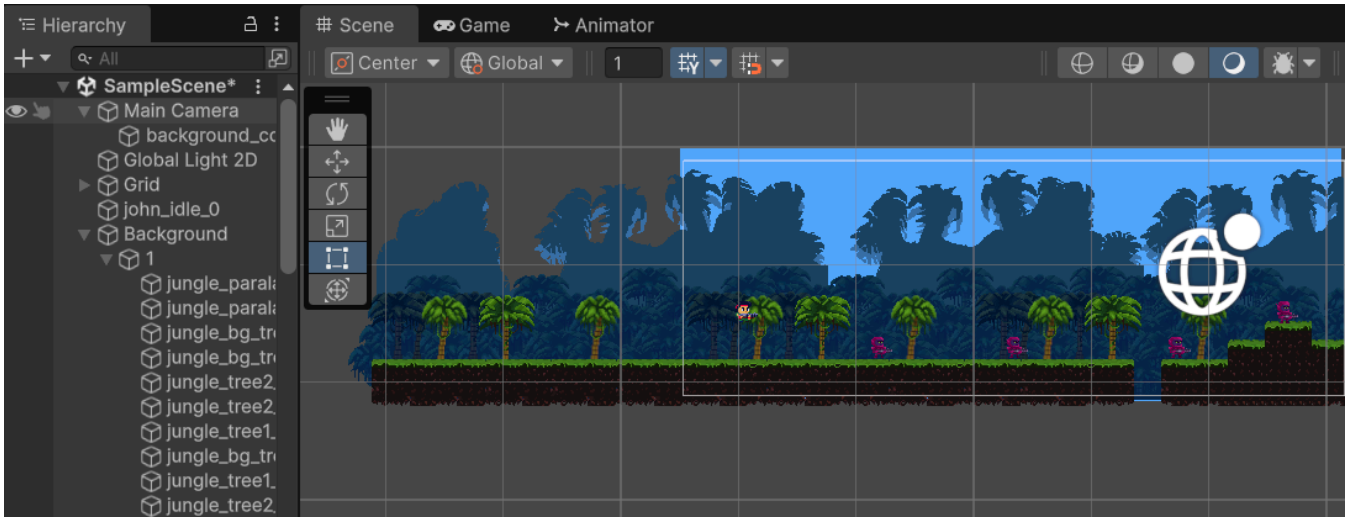
### I. EJERCICIOS PROPUESTOS

**Carpeta con videos:**

https://drive.google.com/drive/folders/145lXy7GcCjjkhQQFxo6mXWNVrOO4AO7F

Para lograr un movimiento continuo del fondo y que solo se mueva el cielo de forma casi estática, hacemos que el objeto (que en este caso es background) sea hijo de la cámara, haciendo que siga el movimiento de esta y la siga de forma continua a lo largo del nivel.

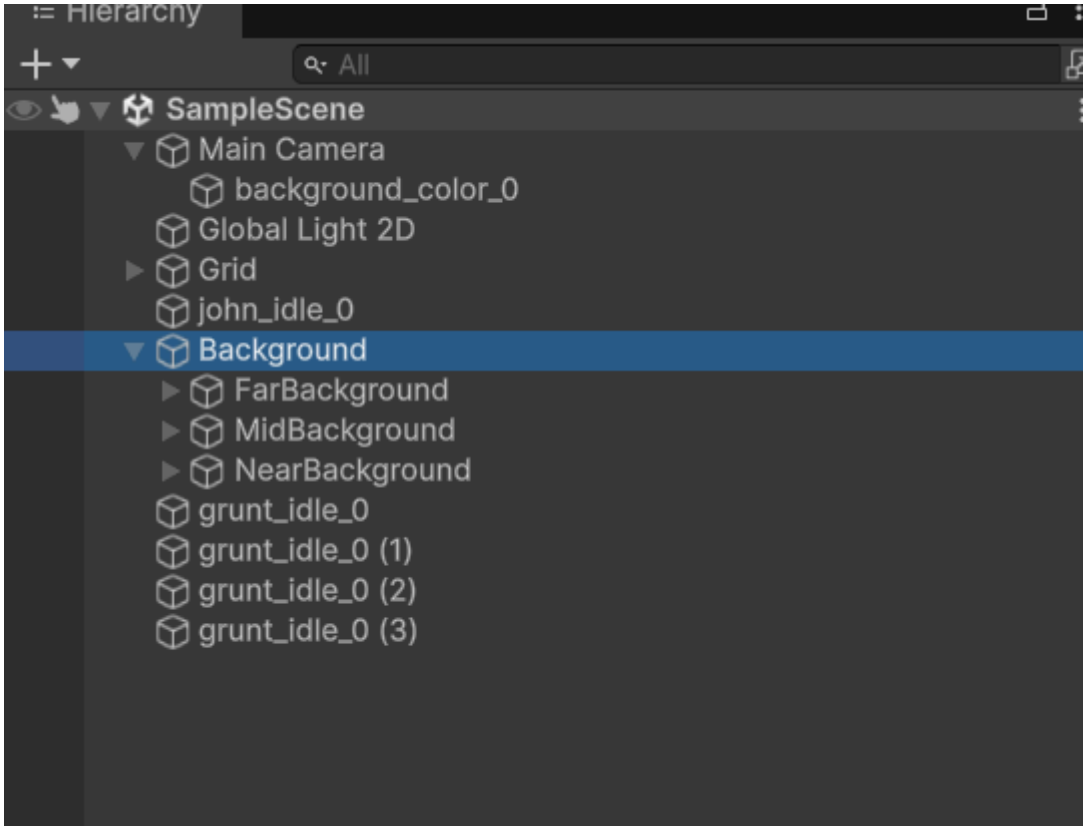
Ideal para eliminar la repetición de muchos objetos background y solo tener uno.




Con esto se cumple la traslación, pues, vamos moviendo el objeto en el eje X en base a las interacciones que haga el usuario, o sea, moverse.

Una vez hecho eso, seguimos con una refactorización en la organización de los elementos en la pantalla, principalmente centrándonos en lo que viene a ser los elementos del campo buscando dividirlos en 3: Lejano, mediano y cercano.

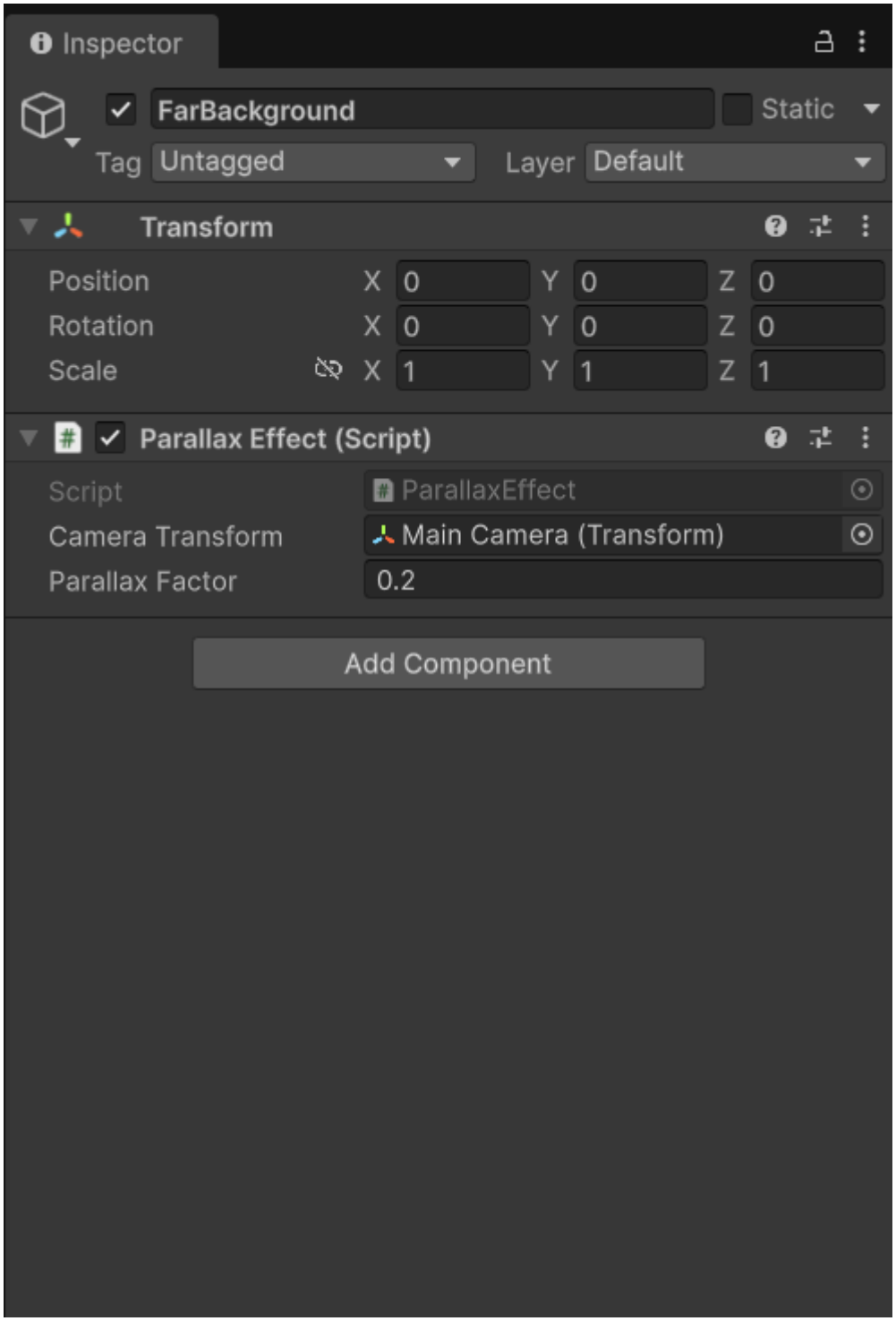
El objetivo de esto es aplicar el ParallaxEffect y lograr que el campo tenga más vida, además de poder hacer ver más limpia la escena.




Agregamos todo a Background.

Luego lo dividimos en 3.

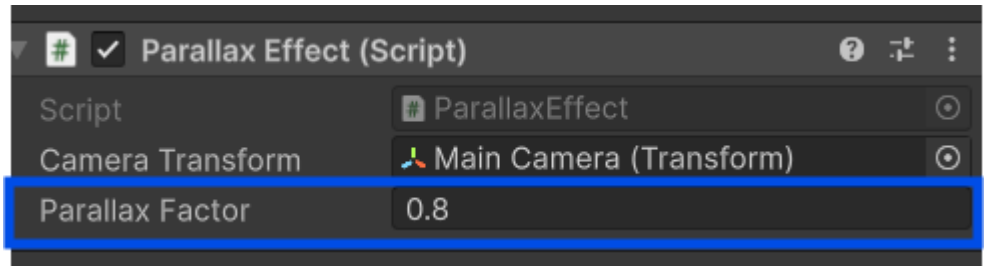
A estos 3 les añadimos el código de ParallaxEffect.




```
C/C++
using UnityEngine;
```

```
public class ParallaxEffect : MonoBehaviour
{
    public Transform cameraTransform;
    public float parallaxFactor; // qué tan rápido se mueve

    private Vector3 lastCameraPosition;

    void Start()
    {
        lastCameraPosition = cameraTransform.position;
    }

    void LateUpdate()
    {
        Vector3 deltaMovement = cameraTransform.position - lastCameraPosition;

        // Solo movemos en X (para juegos 2D horizontales)
        transform.position += new Vector3(deltaMovement.x * parallaxFactor, 0, 0);

        lastCameraPosition = cameraTransform.position;
    }
}
```

Este código se mueve en base a la posición de la cámara y como float ingresamos ParallaxFactor que será la velocidad con la que se moverán los objetos, al hacerlo public podremos modificarlo desde el IDE.
Al agregar un ParallaxFactor, podemos modificar la velocidad conforme vamos probando el juego.




Esto combinado junto con la rotación del player y la transformación que pueden sufrir las balas al chocar, demostraría la implementación de lo solicitado en el laboratorio.

## II. SOLUCIÓN DEL CUESTIONARIO

*No hay un cuestionario*

## III. CONCLUSIONES

- El uso de Parallax permite que un escenario 2D se vea más profundo y dinámico, ya que los fondos se mueven a distintas velocidades según su distancia aparente.

- Esta técnica ayuda a que el jugador sienta mejor el movimiento dentro del nivel, porque no solo se desplaza el personaje, sino también el entorno de una forma más natural.
- En Unity, el Parallax es útil porque mejora bastante la parte visual del juego sin necesidad de usar modelos 3D complejos.
- También permite separar el escenario en capas, como fondo lejano, montañas, árboles o nubes, haciendo que cada elemento tenga una función visual dentro de la escena.
- Una buena implementación de Parallax puede hacer que el nivel se sienta más vivo, aunque el juego siga siendo principalmente 2D.

## RETROALIMENTACIÓN GENERAL

## REFERENCIAS Y BIBLIOGRAFÍA

*[]https://www.youtube.com/watch?v=GbmRt0wydQU*
*[]https://didigameboy.itch.io/jambo-jungle-free-sprites-asset-pack/download/eyJleHBpcmVzIjoxNzc2ODI3NDAxLCJpZCI6MTUxNzE0fQ%3d%3d%2eZOk0YHJ7il1eY2vFT9pM9e1%2fh1M%3d*